

MI PSPI API

Version 1.00

© 2020 SigmaStar Technology Corp. All rights reserved.

SigmaStar Technology makes no representations or warranties including, for example but not limited to, warranties of merchantability, fitness for a particular purpose, non-infringement of any intellectual property right or the accuracy or completeness of this document, and reserves the right to make changes without further notice to any products herein to improve reliability, function or design. No responsibility is assumed by SigmaStar Technology arising out of the application or use of any product or circuit described herein; neither does it convey any license under its patent rights, nor the rights of others.

SigmaStar is a trademark of SigmaStar Technology Corp. Other trademarks or names herein are only for identification purposes only and owned by their respective owners.

REVISION HISTORY

Revision No.	Description	Date
1.00	● Initial release	09/25/2020

TABLE OF CONTENTS

REVISION HISTORY i

TABLE OF CONTENTS ii

1. 概述 1

- 1.1. 模块说明 1
- 1.2. 流程框图 1
- 1.3. 关键字说明 1

2. API 参考 2

- 2.1. API 功能模块 2
- 2.2. MI_PSPI_CreateDevice 2
- 2.3. MI_PSPI_DestroyDevice 3
- 2.4. MI_PSPI_Transfer 3
- 2.5. MI_PSPI_SetDevAttr 4
- 2.6. MI_PSPI_SetOutputAttr 5
- 2.7. MI_PSPI_Enable 6
- 2.8. MI_PSPI_Disable 6

3. PSPI 数据类型 8

- 3.1. 模块相关数据类型定义 8
- 3.2. MI_PSPI_Msg_t 8
- 3.3. MI_PSPI_OutputAttr_t 9
- 3.4. MI_PSPI_Param_t 10
- 3.5. MI_PSPI_DEV 11

4. 程序例程 13

- 4.1. Sensor 例程 13
- 4.2. Panel 例程 15
- 4.3. Sensor 图像由 Panel 显示例程 17

5. 错误码 21

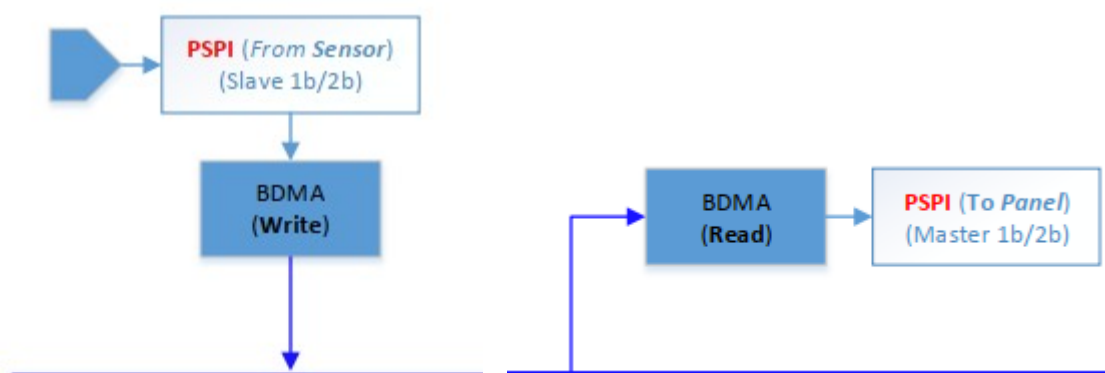
1. 概述

1.1. 模块说明

通过 PSPI 从 sensor 获取图像数据，或将图像数据通过 PSPI 传输到 panel。

注意：目前 sensor 只能接 PSPI0，panel 只能接 PSPI1。

1.2. 流程框图



芯片可以将 PSPI0 设置为从机，然后通过 PSPI 从 sensor 获取数据，同时搭配有一个 BDMA 用来将获取到的数据写入内存；

芯片可以将 PSPI1 设置为主机，然后通过 PSPI 将数据发送到 panel，内部搭配有一个 BDMA 用来读取内存中的数据。

1.3. 关键字说明

无。

2. API 参考

2.1.API 功能模块

API 名	功能
MI_PSPI_CreateDevice	创建 PSPI 设备并配置 PSPI 属性
MI_PSPI_DestroyDevice	销毁 PSPI 设备
MI_PSPI_Transfer	给 PSPI 设备发送参数信息，对外部所接设备的参数进行配置
MI_PSPI_SetDevAttr	设置 PSPI 属性
MI_PSPI_SetOutputAttr	设置 PSPI 输出端口属性
MI_PSPI_Enable	启用 PSPI 设备
MI_PSPI_Disable	禁用 PSPI 设备

2.2.MI_PSPI_CreateDevice

➤ 描述

创建 PSPI 设备并配置其属性。

➤ 语法

```
MI_S32 MI_PSPI_CreateDevice (MI\_PSPI\_DEV PspiDev, MI\_PSPI\_Param\_t * pstPspiParam);
```

➤ 参数

参数名称	描述	输入/输出
PspiDev	PSPI 具体设备	输入
pstPspiParam	PSPI 设备的具体属性	输入

➤ 返回值

返回值 {
MI_PSPI_SUCCESS 成功。
非 MI_PSPI_SUCCESS 失败，具体见[错误码](#)。

➤ 需求

- 头文件：mi_pspi.h、mi_pspi_datatype.h
- 库文件：libmi_pspi.a / libmi_pspi.so

※ 注意

同一 PSPI 设备不能连续创建两次，如果要再修改 PSPI 的属性，可以通过 [MI_PSPI_SetDevAttr](#) 进行修改

2.3.MI_PSPI_DestroyDevice

➤ 描述

销毁 PSPI 设备。

➤ 语法

```
MI_S32 MI_PSPI_DestroyDevice(MI\_PSPI\_DEV PspiDev) ;
```

➤ 参数

参数名称	描述	输入/输出
PspiDev	需要销毁的 PSPI 设备。	输入

➤ 返回值

返回值 {
MI_PSPI_SUCCESS 成功。
非 MI_PSPI_SUCCESS 失败，具体见[错误码](#)。

➤ 需求

- 头文件：mi_pspi.h、mi_pspi_datatype.h
- 库文件：libmi_pspi.a / libmi_pspi.so

2.4.MI_PSPI_Transfer

➤ 描述

给 PSPI 外部所接设备发送参数信息，对外部所接设备的参数进行配置。

➤ 语法

MI_S32 MI_PSPI_Transfer([MI_PSPI_DEV](#) PspiDev, [MI_PSPI_Msg_t](#) *pstMsg)

➤ 参数

参数名称	描述	输入/输出
PspiDev	PSPI 具体设备	输入
pstMsg	发送给从机设备的具体参数	输入

➤ 返回值

返回值 { MI_PSPI_SUCCESS 成功。
非 MI_PSPI_SUCCESS 失败，具体见[错误码](#)。

➤ 需求

- 头文件：mi_pspi.h、mi_pspi_datatype.h
- 库文件：libmi_pspi.a / libmi_pspi.so

※ 注意

- 这个函数是用来通过 PSPI 向从机发送数据来控制从机的，如使用 PSPI 点亮 panel 时，在向 panel 发送具体的图像数据前需要对 panel 发送一些控制参数，这些控制参数就可以通过这个函数进行发送。一次性发送参数的数量可以由 mi_pspi_datatype.h 中的 PSPI_PARAM_BUFF_SIZE 宏来进行控制

2.5.MI_PSPI_SetDevAttr

➤ 描述

配置 PSPI 属性。

➤ 语法

MI_S32 MI_PSPI_SetDevAttr([MI_PSPI_DEV](#) PspiDev, [MI_PSPI_Param_t](#) * pstPspiParam);

➤ 参数

参数名称	描述	输入/输出
PspiDev	PSPI 具体设备	输入
pstPspiParam	即将要进行配置的具体属性	输入

➤ 返回值

{ MI_PSPI_SUCCESS 成功。

返回值

非 MI_PSPI_SUCCESS 失败，具体见[错误码](#)。

➤ 需求

- 头文件：mi_pspi.h、mi_pspi_datatype.h
- 库文件：libmi_pspi.a / libmi_pspi.so

※ 注意

- 这个函数是用来对 PSPI 的属性进行修改，如果不需要修改，可不调用。

2.6.MI_PSPI_SetOutputAttr

➤ 描述

设置 PSPI 输出端口的属性。

➤ 语法

```
MI_S32 MI_PSPI_SetOutputAttr(MI\_PSPI\_OutputAttr\_t * pstOutputAttr);
```

➤ 参数

参数名称	描述	输入/输出
pstOutputAttr	即将要进行配置的属性	输入

➤ 返回值

返回值 { MI_PSPI_SUCCESS 成功。
非 MI_PSPI_SUCCESS 失败,具体见[错误码](#)。

➤ 需求

- 头文件：mi_pspi.h、mi_pspi_datatype.h
- 库文件：libmi_pspi.a / libmi_pspi.so

※ 注意

- 这个函数是用来对存储 sensor 数据的 buffer 属性进行设置，在接 panel 时这个函数无作用。其中 sensor 的输出通道的默认属性为 640 * 480，YUV 422 格式。

2.7.MI_PSPI_Enable

➤ 描述

启用 PSPI 设备。

➤ 语法

```
MI_S32 MI_PSPI_Enable(MI\_PSPI\_DEV PspiDev );
```

➤ 参数

参数名称	描述	输入/输出
PspiDev	要启用的 PSPI 设备	输入

➤ 返回值

返回值 { MI_PSPI_SUCCESS 成功。
非 MI_PSPI_SUCCESS 失败,具体见[错误码](#)。

➤ 需求

- 头文件: mi_pspi.h、mi_pspi_datatype.h
- 库文件: libmi_pspi.a / libmi_pspi.so

※ 注意

- 在使用 mi_sys 的相关接口操作 PSPI 前, 必须调用这个函数进行使能。

2.8.MI_PSPI_Disable

➤ 描述

禁用 PSPI 设备。

➤ 语法

```
MI_S32 MI_PSPI_Disable(MI\_PSPI\_DEV PspiDev );
```

➤ 参数

参数名称	描述	输入/输出
PspiDev	要禁用的 PSPI 设备	输入

➤ 返回值

返回值 { MI_PSPI_SUCCESS 成功。
非 MI_PSPI_SUCCESS 失败,具体见[错误码](#)。

➤ 需求

- 头文件: mi_pspi.h、mi_pspi_datatype.h
- 库文件: libmi_pspi.a / libmi_pspi.so

※ 注意

- 在不需要再使用 PSPI 时, 必须调用这个函数禁用 PSPI。

3. PSPI 数据类型

3.1. 模块相关数据类型定义

数据类型	定义
MI_PSPI_Msg_t	定义传输给外部连接的 PSPI 设备参数的数据帧
MI_PSPI_OutputAttr_t	定义 PSPI 输出端口属性
MI_PSPI_Param_t	定义 PSPI 属性
MI_PSPI_DEV	定义 PSPI 的设备编号

3.2. MI_PSPI_Msg_t

➤ 说明

定义使用 PSPI 向外部设备传输参数时的数据帧。

➤ 定义

```
typedef struct{
    MI_U16  u16TxSize;
    MI_U16  u16RxSize;
    MI_U8   u8TxBitCount;
    MI_U8   u8RxBitCount;
    MI_U16  au16TxBuf[PSPI_PARAM_BUFF_SIZE];
    MI_U16  au16RxBuf[PSPI_PARAM_BUFF_SIZE];
} MI_PSPI_Msg_t ;
```

※ 注意事项

每次传输的最大数据个数是由宏 PSPI_PARAM_BUFF_SIZE 来控制的

➤ 成员

成员名称	描述
u16TxSize	发送数据数目(MI_U16 类型大小)
u16RxSize	接收数据数目(MI_U16 类型大小)
u8TxBitCount	发送时一次传输的 bit 数目
u8RxBitCount	接收时一次传输的 bit 数目
au16TxBuf[PSPI_PARAM_BUFF_SIZE]	发送数据缓冲区
au16RxBuf[PSPI_PARAM_BUFF_SIZE]	接收数据缓冲区

➤ 相关数据类型及接口

PSPI_PARAM_BUFF_SIZE

MI_S32 MI_PSPI_Transfer(MI_PSPI_DEV PspiDev, MI_PSPI_Msg_t *pstMsg)

3.3.MI_PSPI_OutputAttr_t

➤ 说明

定义 PSPI 输出端口属性。由于只有当 PSPI 接 sensor 时才有输出端口，所以这个目前是用来描述所接 sensor 的属性。

➤ 定义

```
typedef struct
{
    MI_SYS_PixelFormat_e ePixelFormat;
    MI_U16 u16Width;
    MI_U16 u16Height;
}MI_PSPI_OutputAttr_t;
```

※ 注意事项

无。

➤ 成员

成员名称	描述
ePixelFormat	定义 sensor 输入数据的格式
u16Width	定义 sensor 输入图像数据的宽
u16Height	定义 sensor 输入图像数据的高

➤ 相关数据类型及接口

MI_S32 [MI_PSPI_SetOutputAttr](#)([MI_PSPI_OutputAttr_t](#) * pstOutputAttr);

3.4.MI_PSPI_Param_t

➤ 说明

定义 PSPI 属性

➤ 定义

typedef struct

{

MI_U32 u32MaxSpeedHz;

MI_U16 u16DelayCycle;

MI_U16 u16WaitCycle;

MI_U16 u16PspiMode;

MI_U8 u8DataLane;

MI_U8 u8BitsPerWord;

MI_U8 u8RgbSwap;

MI_U8 u8TeMode;

MI_U8 u8ChipSelect;

}MI_PSPI_Param_t;

➤ 成员

成员名称	描述	可选值
u32MaxSpeedHz	最大时钟频率	1000000~54000000
u16DelayCycle	SPI_SSCTL 没有设置时，两次传送数据之间的延时	0x0000~0xFFFF，单位为时钟周期
u16WaitCycle	SPI_SSCTL 有设置时，两次传送数据之间的延时	0x0000~0xFFFF，单位为时钟周期
u16PspiMode	PSPI 模式配置	0: 主机模式，MSB，上升沿接收，下降沿发送，片选信号低电平有效，两次数据传送期间片选信号有效。 SPI_CPHA: 上升沿发送，下降沿接收 SPI_CPOL: 上升沿发送，下降沿接收 SPI_CPHA SPI_CPOL: 上升沿接收，下降沿发送。

成员名称	描述	可选值
		SPI_SLAVE: PSPI 作为从机 SPI_LSB: PSPI LSB 先行 SPI_SS POL: 片选信号极性控制, 设置则高电平有效。 SPI_SS CTL: 控制 PSPI 传送两次数据之间片选信号是否继续保持有效。设置则两次数据传送期间片选信号无效。
u8DataLane	发送时的数据线条数	DATA_SINGLE: 单线 DATA_DUAL : 双线 DATA_QUAD : 四线
u8BitsPerWord	每次发送的 bit 数	可以配置为 3~32
u8RgbSwap	给 panel 发送数据时的格式和数据线条数	RGB_SINGLE: RGB 格式, 单线发送 RGB_DUAL: RGB 格式, 双线发送 BGR_SINGLE: BGR 格式, 单线发送 BGR_DUAL: BGR 格式, 双向发送 注意: 需要同步配置 data_lane
u8TeMode	是否使用 TE 模式	1: 使用 TE 模式; 0: 不使用 TE 模式
u8ChipSelect	PSPI 片选信号	MI_PSPI_SELECT_0 MI_PSPI_SELECT_1

※ 注意事项

无。

➤ 相关数据类型及接口

MI_S32 MI_PSPI_CreateDevice(MI_PSPI_DEV PspiDev, MI_PSPI_Param_t * pstPspiParam);

MI_S32 MI_PSPI_SetDevAttr(MI_PSPI_DEV PspiDev, MI_PSPI_Param_t * pstPspiParam);

3.5.MI_PSPI_DEV

➤ 说明

定义芯片内部的 PSPI 编号。

➤ 定义

```
typedef MI_S32 MI_PSPI_DEV;
```

※ 注意事项

这个参数的值可设置为 0 或 1, 分别代表芯片内部的 PSPI 0 和 PSPI1。其中 PSPI0 用来代表接收 (RX), PSPI1 代表发送 (RX)。

➤ 相关数据类型及接口

MI_S32 [MI_PSPI_CreateDevice](#)([MI_PSPI_DEV](#) PspiDev, [MI_PSPI_Param_t](#) * pstPspiParam);

MI_S32 [MI_PSPI_DestroyDevice](#)([MI_PSPI_DEV](#) PspiDev);

MI_S32 [MI_PSPI_Transfer](#)([MI_PSPI_DEV](#) PspiDev, [MI_PSPI_Msg_t](#) * pstMsg);

MI_S32 [MI_PSPI_SetDevAttr](#)([MI_PSPI_DEV](#) PspiDev, [MI_PSPI_Param_t](#) * pstPspiParam);

MI_S32 [MI_PSPI_Disable](#)([MI_PSPI_DEV](#) PspiDev);

MI_S32 [MI_PSPI_Enable](#)([MI_PSPI_DEV](#) PspiDev);

4. 程序例程

4.1.Sensor 例程

```
int main(int argc, char *argv[])
{
    int fd = 0
    MI_PSPI_DEV  pspi_dev = 0;
    MI_PSPI_Param_t pspi_para;
    MI_PSPI_OutputAttr_t stOutputAttr;
    MI_SYS_ChnPort_t stChnPort;
    MI_SYS_BufInfo_t stBufInfo;
    MI_SYS_BUF_HANDLE hSysBuf;

    memset(&stChnPort, 0x0, sizeof(MI_SYS_ChnPort_t));
    memset(&stBufInfo, 0x0, sizeof(MI_SYS_BufInfo_t));
    memset(&hSysBuf, 0x0, sizeof(MI_SYS_BUF_HANDLE));
    memset(&pspi_para, 0x0, sizeof(MI_PSPI_SpiParam_t));

    stChnPort.eModId  = E_MI_MODULE_ID_PSPI;
    stChnPort.u32DevId = 0;
    stChnPort.u32ChnId = 0;
    stChnPort.u32PortId = 0;

    stOutputAttr.u16Width  = 1920;
    stOutputAttr.u16Width  = 1080;
    stOutputAttr.ePixelFormat = E_MI_SYS_PIXEL_FRAME_YUV_SEMIPLANAR_420;

    pspi_para.u8BitsPerWord  = 8;
    pspi_para.u8DataLane     = DATA_DUAL;
```

```

pspi_para.u16DelayCycle    = 0;
    pspi_para.u16WaitCycle    = 0;
pspi_para.u8RgbSwap        = 0;
pspi_para.u32MaxSpeedHz    = 1000000;
pspi_para.u16PspiMode      = SPI_SLAVE;
    pspi_para.u8ChipSelect    = MI_PSPI_SELECT_0;
/**
    通过 I2C 等手段向 sensor 发送命令
***/

MI_SYS_Init();
MI_PSPI_CreateDevice(pspi_dev, &pspi_para);
MI_PSPI_SetOutputAttr(&stOutputAttr); //修改 sensor 输入图像的属性
MI_PSPI_Enable(pspi_dev);//为了减小篇幅去掉了对函数调用的返回值进行判断
MI_SYS_SetChnOutputPortDepth(&stChnPort,3,4);

```

GET_OUT_BUF:

```

if (MI_SUCCESS != MI_SYS_ChnOutputPortGetBuf(&stChnPort, &stBufInfo, &hSysBuf))
{
    goto GET_OUT_BUF;
}

fd = open("picture", O_RDWR|O_CREAT|O_TRUNC, 0777);
write(fd, tBufInfo.stFrameData.pVirAddr[0], stBufInfo.stFrameData.u32BufSize);
sync();
close(fd);

```

PUT_OUT_BUF:

```

if (MI_SUCCESS != MI_SYS_ChnOutputPortPutBuf(hSysBuf))
{
    goto PUT_OUT_BUF;
}

```

```
}
```

```
MI_SYS_SetChnOutputPortDepth(&stChnPort,0,3);
```

```
MI_PSPI_Disable(pspi_dev);
```

```
MI_PSPI_DestroyDevice(pspi_dev);
```

```
MI_SYS_Exit();
```

```
Return 0;
```

```
}
```

4.2.Panel 例程

```
int main (int argc, char *argv[])
```

```
{
```

```
MI_S32 s32Ret = 0;
```

```
MI_U16 * buff = NULL;
```

```
MI_U32 size = 0;
```

```
MI_PSPI_Msg_t pspi_msg;
```

```
MI_PSPI_Param_t pspi_para;
```

```
MI_PSPI_DEV pspi_dev = 1;
```

```
MI_SYS_ChnPort_t stChnPort;
```

```
MI_SYS_BUF_HANDLE stHandle;
```

```
MI_SYS_BufInfo_t stBufInfo;
```

```
MI_SYS_BufConf_t stBufConf;
```

```
pspi_para.u8BitsPerWord = 9;
```

```
pspi_para.u8DataLane = DATA_SINGLE;
```

```
pspi_para.u16DelayCycle = 2;
```

```
pspi_para.u16WaitCycle = 2;
```

```
pspi_para.u8RgbSwap = 0;
```

```
pspi_para.u32MaxSpeedHz = 1000000;
```

```
pspi_para.u16PspiMode = 0;
```

```
pspi_para.u8ChipSelect = MI_PSPI_SELECT_0;
```

```
pspi_para.u8TeMode      = 0;
MI_PSPI_CreateDevice(pspi_dev, &pspi_para); //为了减小篇幅去掉了对函数调用的返回值进行判断
memset(&pspi_msg, 0, sizeof(MI_PSPI_Msg_t));
pspi_msg.u8TxBitCount = 9; //可根据 panel 的不同选择不同的数值
pspi_msg.u8RxBitCount = 8;
pspi_msg.u8TxSize = 1;
//0xDA //根据 panel 的型号发送对应的控制命令，视 panel 型号而定
pspi_msg.au16TxBuf[0] = 0xDA;
MI_PSPI_Transfer(pspi_dev, &pspi_msg);
/**** 向 panel 发送命令 *****/
memset(&stChnPort, 0, sizeof(MI_SYS_ChnPort_t));
memset(&stBufConf, 0, sizeof(MI_SYS_BufConf_t));
memset(&stBufInfo, 0, sizeof(MI_SYS_BufInfo_t));
memset(&stHandle, 0, sizeof(MI_SYS_BUF_HANDLE));

stChnPort.eModId = E_MI_MODULE_ID_PSPI;
stChnPort.u32DevId = 1;
stChnPort.u32ChnId = 0;
stChnPort.u32PortId = 0;

stBufConf.eBufType = E_MI_SYS_BUFDATA_FRAME;
stBufConf.stFrameCfg.u16Height = 240;
stBufConf.stFrameCfg.u16Width = 320; //这些参数视情况而定
stBufConf.stFrameCfg.eFrameScanMode = E_MI_SYS_FRAME_SCAN_MODE_PROGRESSIVE;
stBufConf.stFrameCfg.eFormat = E_MI_SYS_PIXEL_FRAME_RGB565;

MI_PSPI_Enable(pspi_dev);
while(1)
{
    getBuff1:
    if(MI_SYS_ChnInputPortGetBuf(&stChnPort, &stBufConf, &stBufInfo, &stHandle, 4000) != MI_SUCCESS)
```

```

{
    printf("get input port buf red failed\n");
    goto getBuff1;
}
else
{
    printf("MI_SYS_ChnInputPortGetBuf success 1\n");

    buff = (MI_U16 *)stBufInfo.stFrameData.pVirAddr[0];
    size = stBufInfo.stFrameData.u32BufSize/2;
    for(i = 0; i < size ; i++)
    {
        buff[i] = 0xf800;
    }

putbuff1:
    if(MI_SYS_ChnInputPortPutBuf(stHandle, &stBufInfo, FALSE) != MI_SUCCESS)
    {
        printf("writter frame err 1\n");
        goto putbuff1;
    }
    else
        printf("written a frame red to screen success\n");
}
}

```

4.3. Sensor 图像由 Panel 显示例程

(由于当前使用的 sensor 图像的像素和格式与 panel 不同，所以要在两者中间串接一个 divp 模块，divp 模块相关配置请参考 divp 模块文档。)

```

#define FRAME_RATE 30

int main(int argc, char *argv[])
{

```

```
MI_S32 s32Ret = 0;
MI_SYS_ChnPort_t stSensorChnPort;
MI_SYS_ChnPort_t stPanelChnPort;
MI_SYS_ChnPort_t stDivpChnPort;

MI_SYS_BindType_e eBindType;
MI_U32 u32BindParam;
MI_SYS_Init();
/*sensor 初始化并使能，参考 4.1 例程*/
/*panel 初始化并使能，参考 4.2 例程*/

//vpe 初始化
MI_DIVP_ChnAttr_t stAttr;
MI_DIVP_OutputPortAttr_t stOutputPortAttr;
memset(&stAttr, 0, sizeof(stAttr));

stAttr.bHorMirror = false;
stAttr.bVerMirror = false;
stAttr.eDiType = E_MI_DIVP_DI_TYPE_OFF;
stAttr.eRotateType = E_MI_SYS_ROTATE_NONE;
stAttr.eTnrLevel = E_MI_DIVP_TNR_LEVEL_OFF;
stAttr.stCropRect.u16X = 0;
stAttr.stCropRect.u16Y = 0;
stAttr.stCropRect.u16Width = 640;
stAttr.stCropRect.u16Height = 480;
stAttr.u32MaxWidth = 640;
stAttr.u32MaxHeight = 480;

s32Ret = MI_DIVP_CreateChn(0, &stAttr);
stOutputPortAttr.eCompMode = E_MI_SYS_COMPRESS_MODE_NONE;
stOutputPortAttr.ePixelFormat = E_MI_SYS_PIXEL_FRAME_RGB565;
```

```
stOutputPortAttr.u32Width = 240;
stOutputPortAttr.u32Height = 320;
s32Ret = MI_DIVP_SetOutputPortAttr(0, &stOutputPortAttr);
s32Ret = MI_DIVP_StartChn(0);
```

```
stSensorChnPort.eModId = E_MI_MODULE_ID_PSPI;
stSensorChnPort.u32DevId = 0;
stSensorChnPort.u32ChnId = 0;
stSensorChnPort.u32PortId = 0;
```

```
stDivpInputChnPort.eModId = E_MI_MODULE_ID_DIVP;
stDivpInputChnPort.u32DevId = 0;
stDivpInputChnPort.u32ChnId = 0;
stDivpInputChnPort.u32PortId = 0;
```

```
stPanelChnPort.eModId = E_MI_MODULE_ID_PSPI;
stPanelChnPort.u32DevId = 1;
stPanelChnPort.u32ChnId = 0;
stPanelChnPort.u32PortId = 0;
```

```
eBindType = E_MI_SYS_BIND_TYPE_FRAME_BASE;
u32BindParam = 0;
```

```
MI_SYS_SetChnOutputPortDepth(&stSensorChnPort,3,6);
```

```
MI_SYS_BindChnPort2(&stSensorChnPort, &stDivpChnPort, FRAME_RATE, FRAME_RATE, eBindType,
u32BindParam);
```

```
MI_SYS_BindChnPort2(&stDivpChnPort, &stPanelChnPort, FRAME_RATE, FRAME_RATE, eBindType,
u32BindParam);
```

```
While(1)
{
```

```
        sleep(10);  
    }  
  
    MI_PSPI_Disable(0);  
    MI_PSPI_Disable(1);  
    MI_PSPI_DestroyDevice(0);  
    MI_PSPI_DestroyDevice(1);  
  
    return 0;  
}
```


5. 错误码

PSPI API 错误码如下表所示:

错误代码	宏定义	描述
0	MI_PSPI_SUCCESS	函数运行成功
-1	MI_PSPI_FAIL	函数运行失败
-2	MI_ERR_PSPI_NULL_PTR	传入空指针
-3	MI_ERR_PSPI_NO_MEM	没有内存
-4	MI_ERR_PSPI_ILLEGAL_PARAM	传入非法的、不恰当的参数
-5	MI_ERR_PSPI_DEV_NOT_INIT	PSPI 设备没有初始化
-6	MI_ERR_PSPI_ENABLE_CHN_FAILED	使能通道失败
-7	MI_ERR_PSPI_ENABLE_PORT_FAILED	使能端口失败
-8	MI_ERR_PSPI_DISABLE_CHN_FAILED	禁用通道失败
-9	MI_ERR_PSPI_DISABLE_PORT_FAILED	禁用端口失败
-10	MI_ERR_PSPI_DEV_HAVE_INITED	PSPI 设备已经初始化
-11	MI_ERR_PSPI_FAILED_IN_MHAL	在 MHAL 中运行出错